5.3 증명 생성 및 검증 과정 분석

개요

이 문서는 PLONK 예제에서의 증명 생성(Proof Generation)과 검증(Verification) 과정을 상세히 분석합니다. 이 과정은 영지식 증명 시스템의 핵심으로, 증명자가 비밀 정보를 공개하지 않고도 특정 계산을 올바르게 수행했음을 검증자에게 증명하는 과정입니다.

증명 생성 과정 (Proof Generation)

1. 증명 생성 함수 호출

```
let (proof, pi) = {
let  mut  circuit: TestCircuit<BlsScalar, JubJubParameters> = TestCircuit {
a: BlsScalar::from(20u64), // 비밀 입력
b: BlsScalar::from(5u64), // 비밀 입력
c: BlsScalar::from(25u64), // 공개 입력 (20 + 5)
d: BlsScalar::from(100u64), // 공개 입력 (20 * 5)
e: JubJubScalar::from(2u64), // 비밀 입력
f: point_f_pi, // 공개 입력 (GENERATOR * 2)
};
circuit.gen_proof::<PC>(&pp, pk_p, b"Test")
}?;
```

2. gen_proof 메서드 구현

```
fn gen_proof<PC>(
&mut self,
u_params: &PC::UniversalParams,
prover_key: ProverKey<F>,
transcript_init: &'static [u8],
) -> Result<(Proof<F, PC>, PublicInputs<F>), Error> {
// 1. 커밋 키 생성
let circuit_size = self.padded_circuit_size();
let (ck, _) = PC::trim(u_params, circuit_size, 0, None)?;
// 2. Prover 인스턴스 생성
let mut prover = Prover::new(transcript_init);
// 3. 회로 제약 조건 구성 (witness 값들로)
self.gadget(prover.mut_cs())?;
// 4. ProverKey 설정
prover.prover_key = Some(prover_key);
let pi = prover.cs.get_pi().clone();

// 5. 증명 생성
Ok((prover.prove(&ck)?, pi))
}
```

3. 상세 증명 생성 단계 (prove_with_preprocessed)

단계 1: Witness 다항식 계산 및 커밋

```
// 1. Witness 다항식 생성
let  w_l_poly = DensePolynomial::from_coefficients_vec(domain.ifft(w_l_scalar));
let  w_r_poly = DensePolynomial::from_coefficients_vec(domain.ifft(w_r_scalar));
let  w_o_poly = DensePolynomial::from_coefficients_vec(domain.ifft(w_o_scalar));
let  w_4_poly = DensePolynomial::from_coefficients_vec(domain.ifft(w_4_scalar));

// 2. Witness 다항식 커밋
let (w_commits, w_rands) = PC::commit(commit_key, w_polys.iter(), None)?;

// 3. 트랜스크립트에 추가
transcript.append(b"w_l", w_commits[0].commitment());
transcript.append(b"w_r", w_commits[1].commitment());
transcript.append(b"w_o", w_commits[2].commitment());
transcript.append(b"w_4", w_commits[3].commitment());
```

핵심 개념:

- **Witness 다항식**: 회로의 비밀 입력값들을 다항식으로 인코딩
- **IFFT 변환**: 평가 형태에서 계수 형태로 변환하여 커밋 가능하게 함
- **다항식 커밋**: 다항식에 대한 암호학적 커밋 생성

단계 2: 룩업 다항식 생성

```
// 1. 테이블 압축 인수 생성
let zeta = transcript.challenge_scalar(b"zeta");

// 2. 룩업 테이블 압축
let compressed_t_multiset = MultiSet::compress(&[
prover_key.lookup.table_1.clone(),
prover_key.lookup.table_2.clone(),
prover_key.lookup.table_3.clone(),
prover_key.lookup.table_4.clone(),
], zeta);

// 3. 쿼리 다항식 생성
let compressed_f_multiset = MultiSet::compress(&f_scalars, zeta);
let f_poly = DensePolynomial::from_coefficients_vec(
domain.ifft(&compressed_f_multiset.0)
);

// 4. 정렬된 다항식 생성
let (h_1, h_2) = compressed_t_multiset.combine_split(&compressed_f_multiset)?;
```

핵심 개념:

- **룩업 테이블**: 미리 정의된 값들의 테이블
- **쿼리 압축**: 여러 와이어 값을 하나의 값으로 압축

- **정렬 증명**: 쿼리된 값들이 테이블에 존재함을 증명

단계 3: 순열 다항식 계산

```
// 1. 순열 챌린지 생성
let beta = transcript.challenge_scalar(b"beta");
let gamma = transcript.challenge_scalar(b"gamma");

// 2. 순열 다항식 계산
let z_poly = self.cs.perm.compute_permutation_poly(
&domain,
(w_l_scalar, w_r_scalar, w_o_scalar, w_4_scalar),
beta,
gamma,
(&prover_key.permutation.left_sigma.0, /* ... */),
);

// 3. 룩업 순열 다항식 계산
let z_2_poly = DensePolynomial::from_coefficients_slice(
&self.cs.perm.compute_lookup_permutation_poly(
&domain, &compressed_f_multiset.0, &compressed_t_multiset.0,
&h_1.0, &h_2.0, delta, epsilon,
),
```

```
);
```

핵심 개념:

- **순열 증명**: 와이어 간의 연결 관계가 올바름을 증명
- **Grand Product**: 순열 관계를 누적곱으로 표현
- **챌린지 값**: 무작위성을 통한 보안성 확보

단계 4: 몫 다항식 계산

```
// 1. 몫 챌린지 생성
let alpha = transcript.challenge_scalar(b"alpha");
let range_sep_challenge = transcript.challenge_scalar(b"range separation challe
// ... 기타 분리 챌린지들

// 2. 몫 다항식 계산
let t_poly = quotient_poly::compute::<F, P>(
&domain, prover_key, &z_poly, &z_2_poly,
&w_l_poly, &w_r_poly, &w_o_poly, &w_4_poly,
&pi_poly, &f_poly, &table_poly, &h_1_poly, &h_2_poly,
&alpha, &beta, &gamma, &delta, &epsilon, &zeta,
&range_sep_challenge, &logic_sep_challenge,
&fixed_base_sep_challenge, &var_base_sep_challenge,
&lookup_sep_challenge,
);
```

핵심 개념:

- **몫 다항식**: 모든 제약 조건의 선형 결합을 소멸 다항식으로 나눈 결과
- **분리 챌린지**: 서로 다른 제약 조건들을 구분하기 위한 무작위 값
- **제약 조건 집계**: 모든 게이트 제약을 하나의 다항식으로 결합

단계 5: 평가값 계산 및 개방 증명

```
// 1. 평가 챌린지 생성
let z_challenge = transcript.challenge_scalar(b"z");

// 2. 다항식 평가값 계산
let a_eval = w_l_poly.evaluate(&z_challenge);
let b_eval = w_r_poly.evaluate(&z_challenge);
// ... 기타 평가값들

// 3. 개방 증명 생성
let (aw_opening, aw_challenge) = PC::open(
commit_key, &labeled_polynomials, &commitments,
&z_challenge, &mut transcript
)?;
```

핵심 개념:

- **다항식 평가**: 특정 점에서의 다항식 값 계산
- **개방 증명**: 커밋된 다항식이 특정 값을 가짐을 증명

- **배치 개방**: 여러 다항식을 효율적으로 동시에 개방

4. 최종 증명 구조

```rust
pub struct Proof<F, PC> {
// Witness 커밋들
pub(crate) a_comm: PC::Commitment, // w_l 커밋
pub(crate) b_comm: PC::Commitment, // w_r 커밋
pub(crate) c_comm: PC::Commitment, // w_o 커밋
pub(crate) d_comm: PC::Commitment, // w_4 커밋
// 순열 커밋
pub(crate) z_comm: PC::Commitment, // 순열 다항식 커밋
// 몫 다항식 커밋들
pub(crate) t_1_comm: PC::Commitment, // 몫 다항식 조각들
pub(crate) t_2_comm: PC::Commitment,
// ... t_8_comm까지
// 평가값들
pub(crate) a_eval: F, // w_l(z) 평가값
pub(crate) b_eval: F, // w_r(z) 평가값
// ... 기타 평가값들
// 개방 증명들
pub(crate) openings: PC::Proof, // 다항식 개방 증명
pub(crate) batch_proof: Option<PC::BatchProof>,
```

```
}
```

증명 검증 과정 (Proof Verification)

1. 검증 함수 호출

```
let  verifier_data = VerifierData::new(vk, pi);

verify_proof::<BlsScalar, JubJubParameters, PC>(

&pp,

verifier_data.key,

&proof,

&verifier_data.pi,

b"Test",

)?;
```

2. 검증 과정의 주요 단계

단계 1: 트랜스크립트 재구성

```
// 1. 공개 입력 추가

transcript.append(b"pi", pub_inputs);


// 2. Witness 커밋 추가 (증명자와 동일한 순서)

transcript.append(b"w_l", &self.a_comm);
```

```
transcript.append(b"w_r", &self.b_comm);

transcript.append(b"w_o", &self.c_comm);

transcript.append(b"w_4", &self.d_comm);



// 3. 챌린지 재생성

let zeta = transcript.challenge_scalar(b"zeta");

let beta = transcript.challenge_scalar(b"beta");

let gamma = transcript.challenge_scalar(b"gamma");

// ... 기타 챌린지들
```

핵심 개념:

- **트랜스크립트 동기화**: 증명자와 검증자가 동일한 챌린지 생성
- **Fiat-Shamir 변환**: 상호작용을 비상호작용으로 변환
- **챌린지 재생성**: 증명에 포함된 커밋들로부터 챌린지 재계산

단계 2: 제약 조건 검증

```
// 1. 소멸 다항식 평가

let z_h_eval = domain.evaluate_vanishing_polynomial(z_challenge);



// 2. 첫 번째 라그랑주 다항식 평가

let l1_eval = compute_first_lagrange_evaluation(&domain, &z_h_eval, &z_challeng



// 3. 선형화 다항식 계산

let r0 = self.compute_r0(
```

```
&domain, &pub_inputs.as_evals(domain.size()),
alpha, beta, gamma, delta, epsilon, z_challenge, l1_eval,
// ... 기타 평가값들
);
```

핵심 개념:

- **선형화**: 다항식 등식을 선형 결합으로 변환
- **라그랑주 다항식**: 특정 점에서만 1인 다항식
- **제약 조건 집계**: 모든 제약을 하나의 등식으로 결합

단계 3: 다항식 커밋 검증

```
// 1. 선형화 커밋 계산
let lin_comm = self.compute_linearisation_commitment::<P>(
&domain, alpha, beta, gamma, delta, epsilon,
z_challenge, l1_eval, plonk_verifier_key,
);

// 2. 배치 검증 준비
let mut scalars = Vec::new();
let mut points = Vec::new();
// ... 검증할 커밋들과 스칼라들 수집

// 3. 배치 검증 수행
```

```
PC::batch_check(
verifier_key, &commitments, &z_challenge,
&evaluations, &proof.openings, &mut  transcript
)?;
```

핵심 개념:

- **배치 검증**: 여러 다항식 개방을 효율적으로 동시 검증
- **선형화 커밋**: 검증자가 계산할 수 있는 커밋 조합
- **페어링 기반 검증**: 타원곡선 페어링을 통한 다항식 등식 검증

3. 검증 성공 조건

검증이 성공하려면 다음 조건들이 모두 만족되어야 합니다:

1. **커밋 일관성**: 모든 다항식 커밋이 올바른 값에 대응
2. **제약 조건 만족**: 모든 게이트 제약이 올바르게 만족
3. **순열 관계**: 와이어 간 연결이 올바르게 구성
4. **룩업 관계**: 모든 룩업이 테이블에 존재하는 값
5. **개방 증명**: 모든 다항식 평가값이 올바름

예제에서의 구체적 검증

simple_circuit.rs 예제의 검증 과정

```
// 입력값들
a = 20, b = 5, c = 25, d = 100, e = 2, f = GENERATOR * 2
```

```
// 검증되는 제약 조건들:

1. a + b = c → 20 + 5 = 25 ✓

2. a ≤ 2^6 → 20 ≤ 64 ✓

3. b ≤ 2^4 → 5 ≤ 16 ✓

4. a * b = d → 20 * 5 = 100 ✓

5. GENERATOR * e = f → GENERATOR * 2 = f ✓
```

검증 과정:

1. 증명자가 제공한 평가값들이 실제 제약을 만족하는지 확인
2. 다항식 커밋들이 해당 평가값들과 일치하는지 검증
3. 순열 관계가 올바르게 구성되었는지 확인
4. 모든 암호학적 증명들이 유효한지 검증

성능 및 보안 고려사항

1. 성능 최적화

- **배치 검증**: 여러 다항식을 동시에 검증하여 효율성 향상
- **전처리**: 컴파일 시 미리 계산된 값들 활용
- **FFT 최적화**: 2의 거듭제곱 크기 도메인 사용

2. 보안 속성

- **영지식성**: 비밀 입력값들이 노출되지 않음
- **건전성**: 거짓 명제에 대한 증명 생성 불가능
- **완전성**: 참 명제에 대한 증명은 항상 검증 통과

3. 무작위성과 챌린지

- **Fiat-Shamir**: 해시 함수를 통한 무작위 챌린지 생성
- **챌린지 분리**: 서로 다른 제약 조건들의 독립성 보장
- **트랜스크립트 보안**: 모든 상호작용이 암호학적으로 안전

요구사항 충족 분석

요구사항 5.3 충족도

- ✅ **예제에서의 증명 생성 과정 파악**:
- Witness 다항식 생성 및 커밋
- 룩업 및 순열 다항식 계산
- 몫 다항식 계산 및 평가값 생성
- 개방 증명 생성
- ✅ **예제에서의 증명 검증 과정 파악**:
- 트랜스크립트 재구성 및 챌린지 재생성
- 제약 조건 검증 및 선형화
- 다항식 커밋 배치 검증
- 최종 검증 결과 확인

결론

PLONK의 증명 생성 및 검증 과정은 다음과 같은 특징을 가집니다:

1. **체계적 구조**: 명확하게 정의된 단계별 프로세스
2. **효율적 설계**: 배치 처리와 전처리를 통한 성능 최적화

3. **강력한 보안**: 다중 계층의 암호학적 보안 메커니즘

4. **확장 가능성**: 다양한 제약 조건과 게이트 타입 지원

5. **실용적 구현**: 실제 사용 가능한 성능과 크기

이러한 설계를 통해 PLONK는 복잡한 계산에 대한 효율적이고 안전한 영지식 증명을 제공합니다.